

Documentation of an Autonomous Vehicle Intersection Management System

Moshiur Rahman Prince
Electronic Engineering
Hochschule Hamm-Lippstadt
Hamm, Germany
moshiur-rahman.prince@stud.hshl.de

Md Ratul Ahmed Alvi
Electronic Engineering
Hochschule Hamm-Lippstadt
Hamm, Germany
md-ratul-ahmed.alvi@stud.hshl.de

Md. Tawhidur Rahman Tuhin
Electronic Engineering
Hochschule Hamm-Lippstadt
Hamm, Germany
md-tawhidur-rahman.tuhin@stud.hshl.de

Abstract—This laboratory documentation presents the design, formal verification, software implementation, hardware/software co-design, and testing of AVIMS, an Autonomous Vehicle Intersection Management System developed for Embedded Electronic Engineering Lab A. The project models a signal-free four-way intersection in which autonomous vehicles request permission from a roadside coordination server. A First-In-First-Served queue and a twelve-movement conflict matrix determine safe admission. Two service configurations are compared: an M/M/1-style configuration that admits one vehicle at a time and an M/M/2-style configuration that may admit two compatible movements concurrently. The engineering workflow connects requirements and behavioural models, UPPAAL timed automata, seven formal verification properties, a portable C scheduling core, FreeRTOS tasks, a browser-based live demonstration, and a VHDL conflict ROM verified through a self-checking testbench. All figures in this report are extracted directly from the team's submitted presentation artefacts; no technical diagrams or waveform images are reconstructed.

Index Terms—autonomous vehicles, intersection management, UPPAAL, FreeRTOS, VHDL, ModelSim, HW/SW co-design, formal verification

I. INTRODUCTION AND LABORATORY OBJECTIVE

The project investigates how autonomous vehicles can safely and efficiently share a four-way intersection without conventional traffic lights. Vehicles communicate their position and intended movement over vehicle-to-infrastructure links [4], while a roadside controller determines when each vehicle may enter. The controller must prevent conflicting trajectories from occupying the intersection simultaneously, preserve First-In-First-Served (FIFS) ordering, satisfy a 50 ms decision-time bound, and synchronize state with a backup server before granting permission. The work addresses three laboratory questions posed by the assignment [1]: how intersection traffic can be represented using queuing theory, which scheduling strategy better reduces congestion, and how the concept can be implemented using C and FreeRTOS with a selected function moved to VHDL hardware. The resulting workflow is:

Requirements → System Models → UPPAAL
→ C/FreeRTOS → VHDL/Test.

II. TECHNICAL BACKGROUND AND RELATED WORK

Autonomous intersection management replaces coarse traffic-light phases with finer-grained coordination of individual vehicle movements. Reservation-based approaches demonstrated

that autonomous vehicles can request space–time access to an intersection and be admitted only when the reservation is compatible with existing reservations [2], [3]. AVIMS follows the same broad safety principle but implements it as a compact laboratory architecture based on twelve discrete movements and a shared conflict matrix. Real-time correctness is addressed with timed automata. Timed automata extend finite-state machines with clocks, invariants, and guards, enabling explicit reasoning about deadlines and bounded durations [5]. UPPAAL provides modelling, simulation, and symbolic verification for networks of timed automata [6]–[8]. In AVIMS, this is used to verify collision freedom, permission discipline, bounded decision time, synchronization ordering, occupancy limits, and reachability of safe concurrency. The executable implementation uses FreeRTOS mechanisms for task scheduling and communication [12], [15]. The deterministic pairwise conflict test is additionally implemented in VHDL as a ROM-style lookup, illustrating HW/SW co-design by retaining dynamic scheduling policy in software while mapping a compact repeated decision to hardware [16], [17].

III. REQUIREMENTS ENGINEERING

The requirements model decomposes the overall system goal into functional and quality requirements. The processing pipeline includes vehicle detection, state exchange, FIFS queuing, trajectory conflict detection, scheduling, authorization, and exit monitoring. Safety requires zero conflicting simultaneous crossings in normal operation. Timing requires each scheduling decision to remain within 50 ms. Reliability is supported by primary–backup coordination. A major design principle is that the coordination servers belong to the system itself rather than being treated as external actors. External participants include autonomous vehicles, nearby vehicles involved in V2V communication, and roadside sensing infrastructure.

IV. USE-CASE MODEL

The use-case diagram describes the external interactions and the internal decision chain. The autonomous vehicle registers and exchanges state, requests crossing permission, crosses after authorization, and is monitored until exit. The internal chain connects permission granting to scheduling, conflict detection, FIFS ordering, and timing constraints.

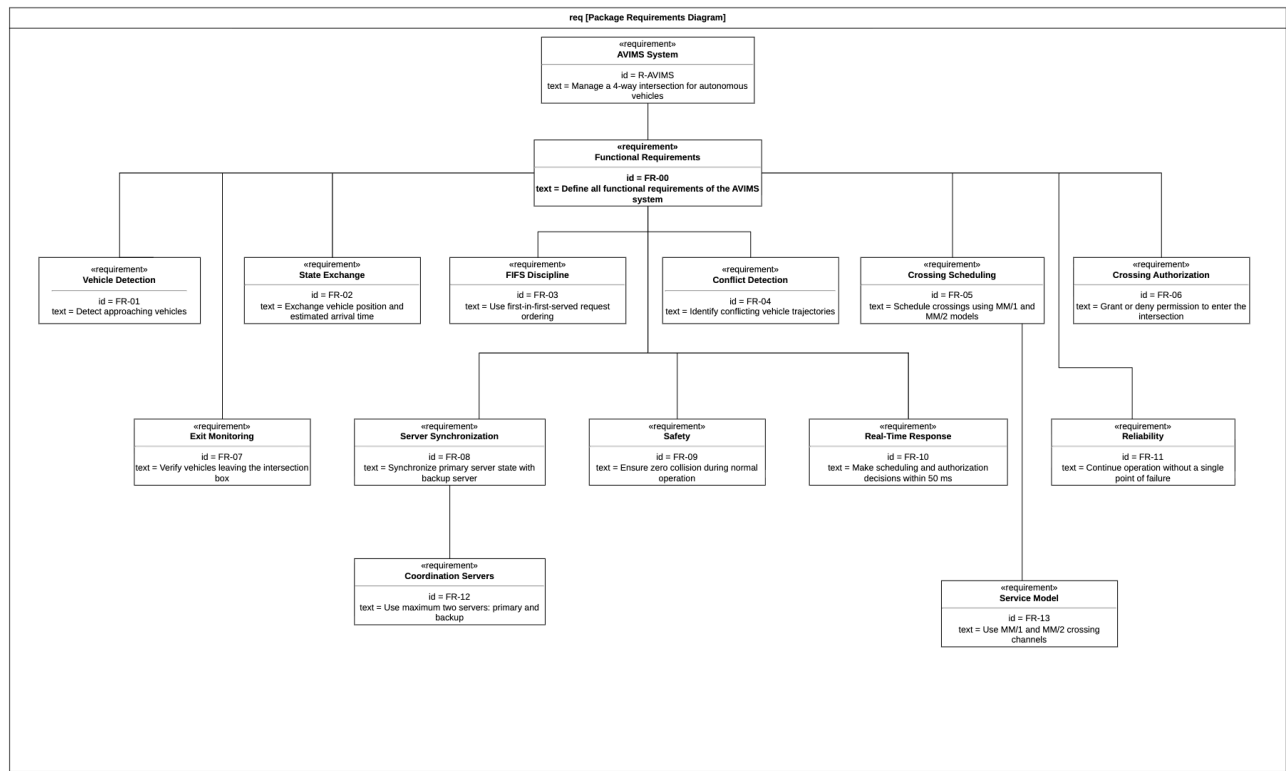


Fig. 1. Requirements diagram extracted directly from the team presentation. The model establishes the traceability chain from vehicle detection and communication to scheduling, safety, timing, and fault tolerance.

V. ACTIVITY MODEL

The activity diagram follows one request through the complete control flow. After approach detection and state exchange, the primary server inserts the request into FIFO order and checks the requested movement against vehicles already committed to the intersection. The request is evaluated under the selected one- or two-channel service configuration. Before a grant is issued, the current decision state is synchronized with the backup server. If a request cannot be safely granted, it remains waiting and returns to a later decision cycle rather than being discarded. When a vehicle is granted permission, it crosses and its exit releases capacity for later requests.

VI. SEQUENCE MODEL

The sequence model makes message ordering explicit. A request is enqueued, its trajectory is evaluated, conflicts are checked, and a schedule is calculated. The primary server then sends state to the backup and waits for acknowledgement before releasing a grant. The model distinguishes three outcomes: denial when a conflict exists, a single crossing under the one-channel case, and simultaneous admission of a compatible pair under the two-channel case.

VII. UPPAAL FORMAL MODEL

The UPPAAL model is a network of six timed automata instantiated from three template types: four vehicles, one pri-

mary server, and one backup server. Vehicles progress through Idle, Waiting, Crossing, and Done. The vehicle crossing time is bounded between 20 and 80 ms through a clock invariant and transition guard. The primary server progresses through Idle, Deciding, synchronization, and Granting behaviour. Its decision clock enforces the 50 ms real-time bound. The backup server receives synchronization state and returns an acknowledgement. The primary cannot release the grant until the handshake is complete. The presentation also contains additional vehicle instances, confirming that the model is instantiated for multiple cars operating in parallel.

VIII. FORMAL VERIFICATION

Seven properties are used to verify the model. UPPAAL supports reachability, safety, liveness-style, and leads-to reasoning over timed executions [7], [9]:

- 1) the system is deadlock-free;
- 2) conflicting movements are never simultaneously inside the intersection;
- 3) no vehicle is inside without permission;
- 4) every scheduling decision respects the 50 ms timing budget;
- 5) backup synchronization occurs before active permission;
- 6) occupancy never exceeds the configured number of channels;

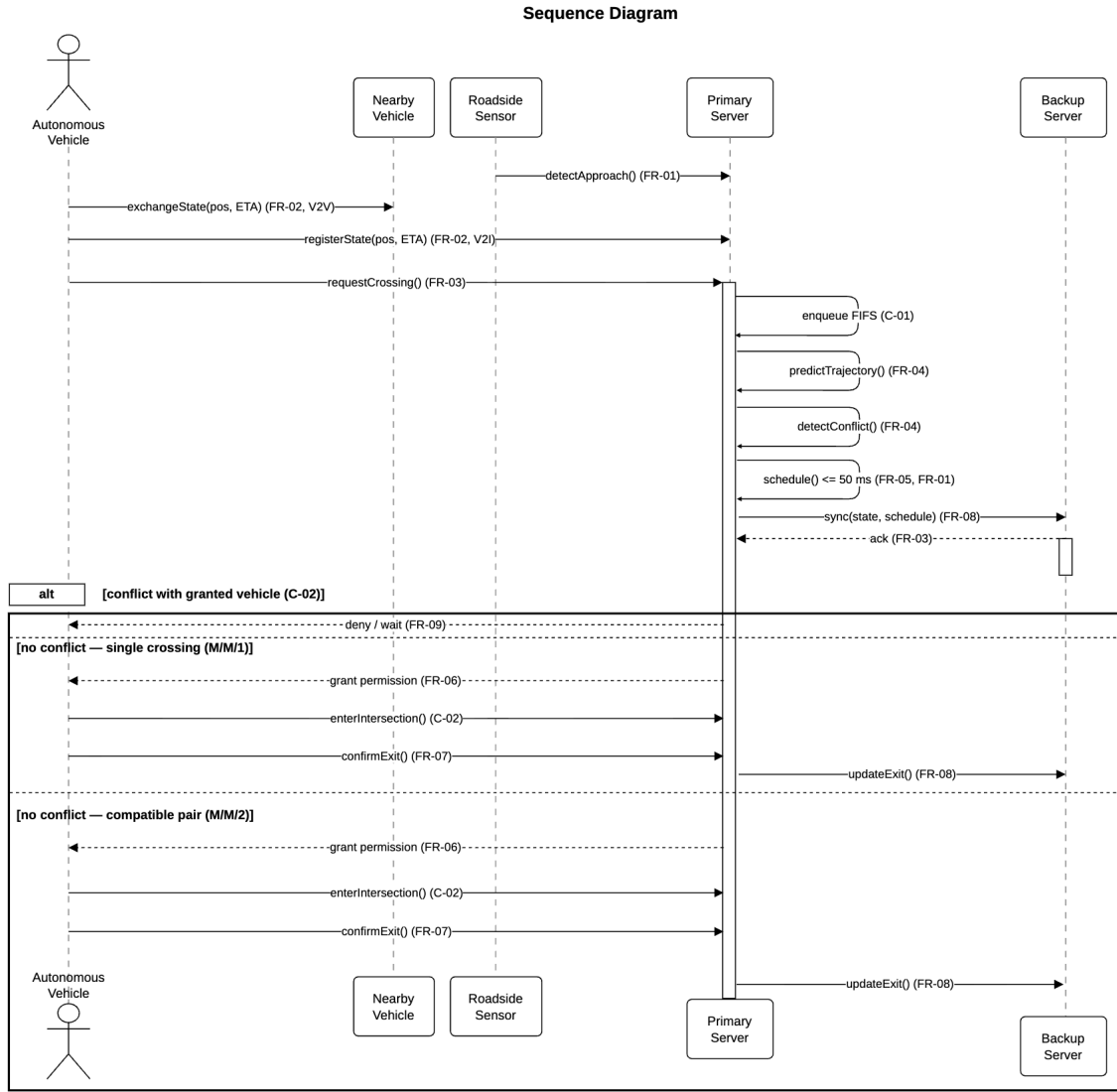
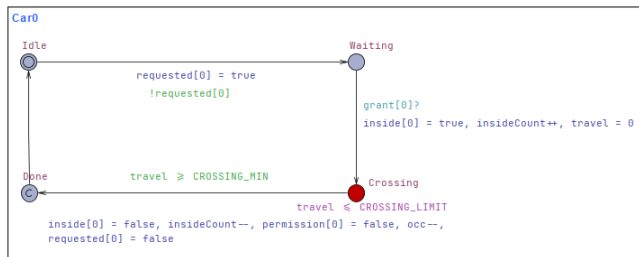
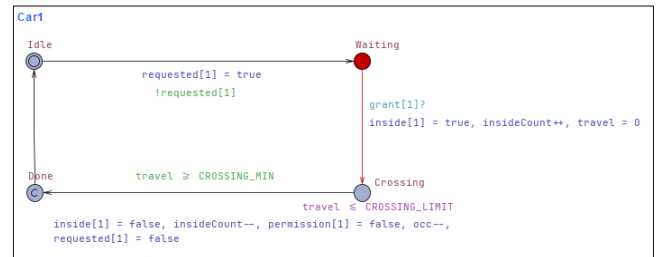


Fig. 4. Sequence diagram extracted from the presentation. The synchronization and acknowledgement occur before the crossing grant.



(a) Vehicle instance



(b) Vehicle instance with different movement assignment

Fig. 5. Vehicle timed automata extracted directly from the uploaded presentation.

at a time, even when another waiting movement would be compatible. This produces strict serialization. In the two-channel configuration, a second vehicle may be admitted only when the conflict matrix confirms compatibility with every already committed movement. For example, North-

Straight and South-Straight may proceed together, while North-Straight and East-Straight conflict in the intersection centre. The project uses the terms M/M/1 and M/M/2 according to the laboratory framing. Strictly, an M/M/c queue assumes Poisson arrivals and exponentially distributed service times;

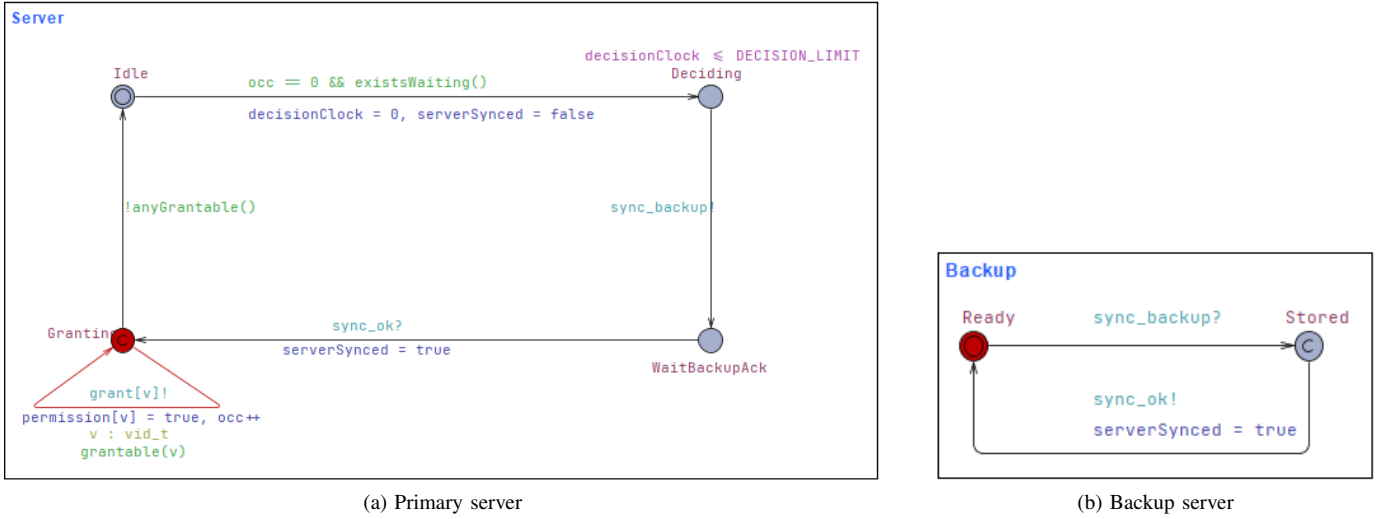


Fig. 6. Primary and backup coordination-server timed automata extracted from the presentation.

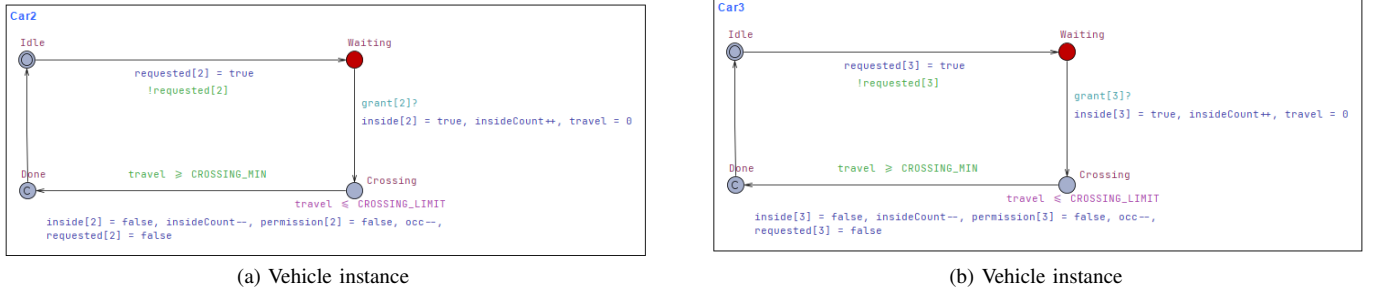


Fig. 7. Further vehicle automata from the presentation, illustrating parallel instantiation of the common vehicle template.

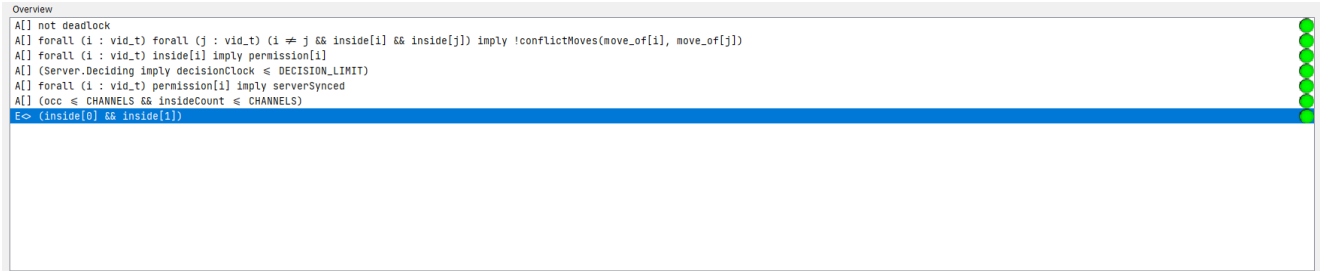


Fig. 8. UPPAAL verification evidence extracted directly from the team presentation.

therefore, the implemented comparison is most accurately described as one-channel versus two-channel conflict-aware service unless those stochastic assumptions are explicitly generated and measured [10], [11]. Strictly speaking, the current implementation demonstrates one- and two-channel service behaviour; a complete stochastic M/M/c experiment would additionally require measured or generated exponential arrival and service-time distributions.

X. CONFLICT MATRIX AS THE SINGLE SOURCE OF TRUTH

Four approaches with three turn choices produce twelve movement classes. Therefore, pairwise compatibility is represented by a 12×12 matrix:

$$C(i, j) = \begin{cases} 1, & \text{movements } i \text{ and } j \text{ conflict,} \\ 0, & \text{otherwise.} \end{cases}$$

For a candidate movement m and committed set A , safe admission requires

$$\forall a \in A : C(m, a) = 0.$$

Table I lists the full 12×12 matrix as generated by the project's conflict-table generator (`tools/gen_conflict.py`) and shared, byte-for-byte identical, across the UPPAAL model, the C header, and the VHDL ROM.

TABLE I
TRAJECTORY CONFLICT MATRIX (1 = CONFLICT, 0 = COMPATIBLE)

	NR	NS	NL	ER	ES	EL	SR	SS	SL	WR	WS	WL
NR	0	1	1	0	1	0	0	0	1	0	0	0
NS	1	0	1	0	1	1	0	0	1	1	1	1
NL	1	1	0	0	1	1	1	1	0	0	1	1
ER	0	0	0	0	1	1	0	1	0	0	0	1
ES	1	1	1	1	0	1	0	1	1	0	0	1
EL	0	1	1	1	1	0	0	1	1	1	1	0
SR	0	0	1	0	0	0	0	1	1	0	1	0
SS	0	0	1	1	1	1	1	0	1	0	1	1
SL	1	1	0	0	1	1	1	1	0	0	1	1
WR	0	1	0	0	0	1	0	0	0	0	1	1
WS	0	1	1	0	0	1	1	1	1	1	0	1
WL	0	1	1	1	1	0	0	1	1	1	1	0

The engineering importance of this matrix is cross-layer consistency. The project maintains corresponding representations in documentation, UPPAAL, the C header, and the VHDL ROM. A generator and consistency-checking approach is used to avoid independent handwritten copies silently diverging.

XI. PORTABLE C CORE AND FREERTOS ARCHITECTURE

The scheduling logic is separated into a portable C core with no direct operating-system dependency. The surrounding implementation uses the FreeRTOS real-time kernel, whose mechanisms include tasks, queues, semaphores/mutexes, and direct-to-task notifications [12]–[15]. The core maintains FIFS queue state, committed movements, service-channel capacity, and the grant rule. This permits isolated unit testing before introducing RTOS scheduling effects. Around the core, the FreeRTOS implementation uses four functional task types. The *SchedulerTask* has the highest application priority and is the sole owner of scheduling state. The *BackupTask* stores synchronization state and returns acknowledgements. The *CommandTask* converts non-blocking user commands into queue messages. Four *CrossWorker* tasks time vehicle crossings and report exit events. Communication uses FreeRTOS queues, direct task notifications, a mutex for clean event output, and *vTaskDelay* for pausable crossing timing. Queues provide FIFO inter-task communication, while task notifications provide lightweight direct event delivery to a task [13], [14]. A vehicle or worker cannot enter merely because it requested access; the scheduler must explicitly grant it.

XII. LIVE DEMONSTRATION ARCHITECTURE

The browser demonstration is intentionally separated from scheduling. The browser renders state but does not make admission decisions. A lightweight Python bridge forwards text commands to the FreeRTOS engine through standard input and streams JSON events back to the browser through Server-Sent Events (SSE) [19].

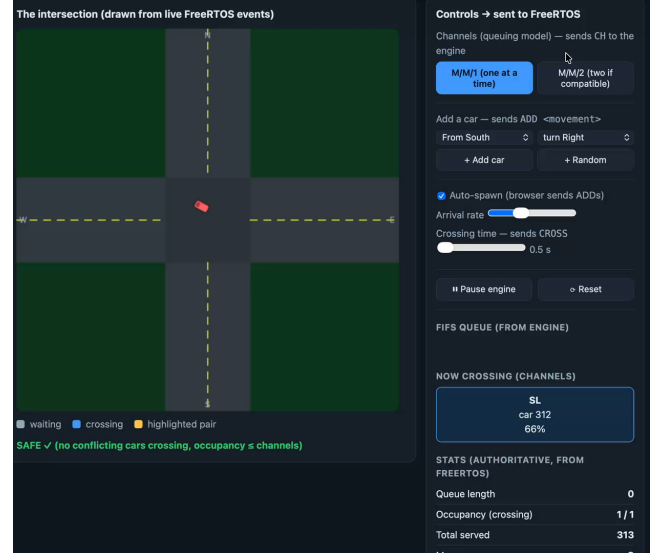


Fig. 9. Actual live browser interface extracted directly from the presentation.

The live demonstration supports commands for adding requests, switching channel count, changing crossing timing, pausing, and resetting. The intended demonstration scenario shows compatible North-Straight and South-Straight movements crossing concurrently while a conflicting East-Straight request remains waiting.

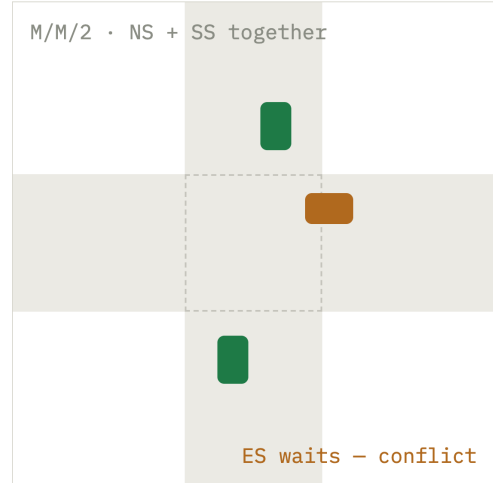


Fig. 10. Actual M/M/2 demonstration visual extracted from the presentation: compatible movements proceed together while a conflicting request waits.

XIII. HW/SW CO-DESIGN

The repeated operation “do movement A and movement B conflict?” was selected for hardware implementation. Partitioning computational functions between software and dedicated hardware is a central HW/SW co-design technique [16]. Dynamic queue management and scheduling policy remain in software, while pairwise conflict lookup is implemented in *conflict_rom.vhd*. The hardware receives two movement identifiers and returns the corresponding conflict bit. This is an appropriate partition because the lookup is deterministic, compact, frequently executed, and naturally represented as

combinational hardware. The software remains responsible for flexible policy, FIFO queue management, backup synchronization, and task coordination.

XIV. VHDL TESTBENCH AND MODELSIM/QUESTASIM EVIDENCE

The VHDL testbench checks the conflict ROM over all $12 \times 12 = 144$ movement pairs. VHDL is standardized by IEEE Std 1076, while HDL testbenches and waveform simulation are standard verification techniques in FPGA development flows [17], [18]. In addition to the complete sweep, it checks properties such as diagonal behaviour, symmetry, known compatible pairs, and known conflicts. The testbench is self-checking and reports success only when the expected and observed results agree. Crucially, the figures below are not reconstructed waveforms. They are the actual simulation images embedded in the team's uploaded presentation, produced using AMD Vivado's HDL simulation and third-party simulator workflows [18]. The waveform evidence supports the hardware-validation claim that the ROM responds consistently to the tested movement identifiers. For final submission, the accompanying text should preserve the exact simulator version and final transcript message used during the team's run.

XV. TESTING AND VERIFICATION STRATEGY

AVIMS is tested at multiple abstraction levels. The UPPAAL model verifies temporal and safety properties over the reachable abstract state space. The portable C core is unit-tested independently of FreeRTOS. The FreeRTOS implementation validates task coordination, queues, notifications, timing, and primary-backup ordering. The VHDL testbench validates all 144 conflict lookups. Cross-layer consistency checking confirms that the formal, software, and hardware conflict representations agree. This layered approach is stronger than relying on one simulation alone because each level addresses a different failure class. Formal verification addresses reachable-state safety and timing properties; C testing addresses scheduler logic; RTOS testing addresses concurrency and communication; and the VHDL testbench addresses hardware lookup correctness.

XVI. TRACEABILITY

Table II, given at the end of this document after the references, summarizes the cross-layer traceability for six representative requirements, following each one from the system model through the UPPAAL formal model to its C/FreeRTOS and VHDL/test counterpart.

XVII. RESULTS AND DISCUSSION

The project demonstrates that the same intersection-management concept can be traced from requirements through formal modelling, software execution, and hardware implementation. The one-channel mode strictly serializes crossings. The two-channel mode permits a second vehicle only when the shared conflict matrix confirms compatibility. The UPPAAL witness property demonstrates reachability of safe two-vehicle concurrency, while the live demonstration visualizes the corresponding runtime behaviour. The strongest engineering result is cross-layer agreement. The conflict relation is not merely

described in the report; it is used by the formal model, C implementation, VHDL hardware, and visualization. Likewise, the backup-before-grant rule appears in the sequence model, UPPAAL server automata, and FreeRTOS queue handshake. The current project remains a laboratory prototype. It does not yet include real V2X radio hardware, localization uncertainty, packet loss, cryptographic authentication, emergency-vehicle priority, mixed human/autonomous traffic, or physical braking dynamics. In addition, quantitative congestion claims should ideally be supported by repeated experiments reporting average waiting time, throughput, queue length, and utilization.

XVIII. RECOMMENDED QUANTITATIVE EVALUATION

A final experimental campaign should compare both channel configurations using identical request sequences. Average waiting time can be calculated as

$$\bar{W} = \frac{1}{N} \sum_{k=1}^N (t_{\text{grant},k} - t_{\text{request},k}),$$

while throughput is

$$\lambda_{\text{out}} = \frac{N_{\text{completed}}}{T}.$$

The experiment should additionally record maximum queue length, channel utilization, concurrent-grant count, and timing-bound violations. This would provide numerical evidence for the congestion comparison.

XIX. TEAM CONTRIBUTION AND PAPER-WRITING SPLIT

The AVIMS project was developed collaboratively by Moshior Rahman Prince, Md Ratul Ahmed Alvi, and Md Tawhidur Rahman Tuhin. All three members were involved in the overall project, including requirements, modelling, conflict-matrix decisions, implementation, testing, debugging, integration, verification, and final review. To make responsibilities clear and to satisfy the requirement that each student contributes at least two to three pages of written technical content, the main drafting responsibility for the paper was divided as shown in Table III. The remaining parts of the paper, especially traceability, results and discussion, contribution review, and conclusion, were jointly reviewed and finalized by all three members. Table III, given at the end of this document after the references, shows the main responsibility areas and the primary writing split; it does not mean that the project was developed as three isolated parts. For the oral presentation only, the speaking parts were divided for a clear delivery: Md Tawhidur Rahman Tuhin presented the system-modelling and UML section, Moshior Rahman Prince presented the UPPAAL model and verification results, and Md Ratul Ahmed Alvi presented the FreeRTOS implementation, live demonstration, and HW/SW co-design. This speaking split was only for the presentation and does not represent exclusive ownership of the development work.

XX. CONCLUSION

AVIMS provides a complete laboratory workflow for autonomous intersection coordination. Requirements and behavioural models define the intended system; UPPAAL verifies

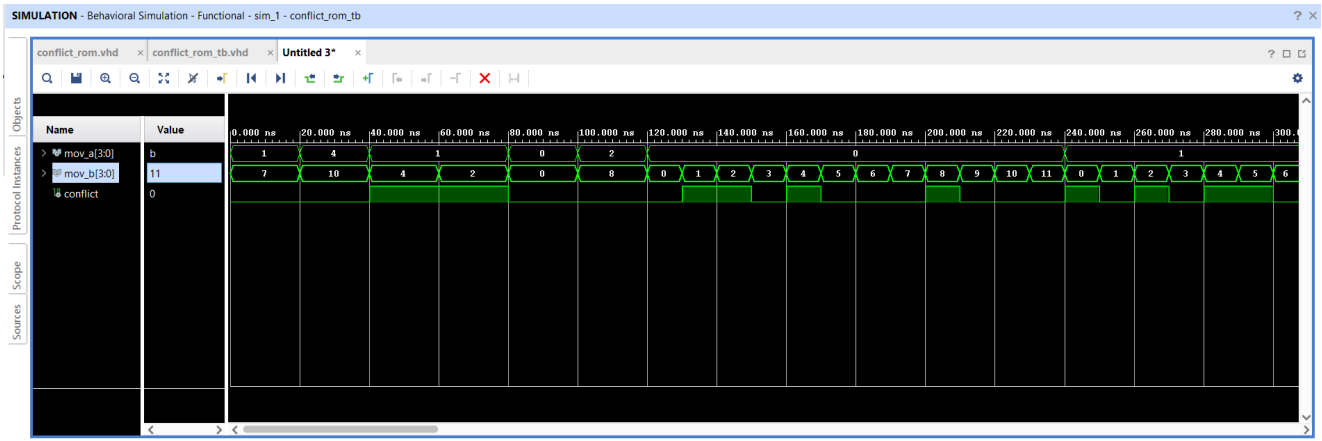


Fig. 11. Actual ModelSim/QuestaSim waveform evidence extracted directly from the presentation.

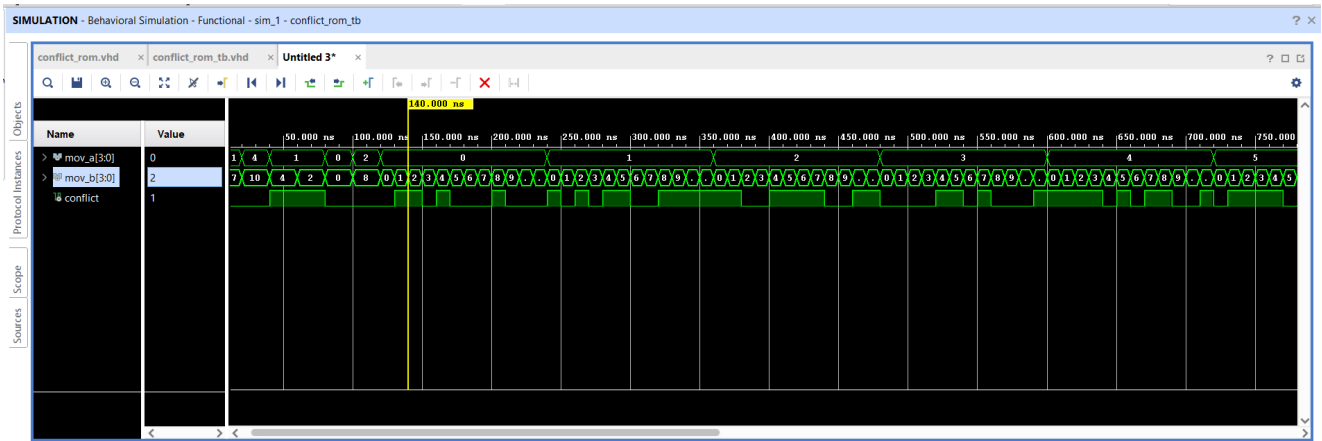


Fig. 12. Additional actual ModelSim/QuestaSim waveform evidence extracted directly from the presentation.

safety, permission, timing, occupancy, synchronization, and concurrency properties; a portable C core implements the scheduling rule; FreeRTOS realizes concurrent task behaviour; and VHDL implements the conflict lookup as hardware. Actual presentation artefacts, including UML diagrams, UPPAAL automata, verifier output, conflict-matrix visualization, live-demo screenshots, and ModelSim/QuestaSim waveform evidence, are included in this documentation. The result is a traceable HW/SW co-design project in which the same core safety decision is represented and checked across modelling, software, and hardware layers.

REFERENCES

- [1] A. Rettberg and S. Henkler, "Embedded Electronic Engineering Lab A: Cross-Traffic Management Project Assignment," Hochschule Hamm-Lippstadt, course material, May 2026.
- [2] K. Dresner and P. Stone, "Multiagent traffic management: A reservation-based intersection control mechanism," in *Proc. AAMAS*, 2004, pp. 530–537.
- [3] K. Dresner and P. Stone, "A multiagent approach to autonomous intersection management," *J. Artificial Intelligence Research*, vol. 31, pp. 591–656, 2008.
- [4] J. B. Kenney, "Dedicated short-range communications (DSRC) standards in the United States," *Proc. IEEE*, vol. 99, no. 7, pp. 1162–1182, Jul. 2011.
- [5] R. Alur and D. L. Dill, "A theory of timed automata," *Theoretical Computer Science*, vol. 126, no. 2, pp. 183–235, 1994.
- [6] K. G. Larsen, P. Pettersson, and W. Yi, "UPPAAL in a nutshell," *Int. J. Software Tools for Technology Transfer*, vol. 1, nos. 1–2, pp. 134–152, 1997.
- [7] G. Behrmann, A. David, and K. G. Larsen, "A tutorial on UPPAAL," in *Formal Methods for the Design of Real-Time Systems*, LNCS 3185. Berlin, Germany: Springer, 2004, pp. 200–236.
- [8] UPPAAL, "Online documentation," accessed Jul. 15, 2026.
- [9] UPPAAL, "Semantics of the symbolic queries," Language Reference, accessed Jul. 15, 2026.
- [10] D. G. Kendall, "Stochastic processes occurring in the theory of queues and their analysis by the method of the imbedded Markov chain," *Ann. Math. Statist.*, vol. 24, no. 3, pp. 338–354, 1953.
- [11] L. Kleinrock, *Queueing Systems, Volume 1: Theory*. New York: Wiley, 1975.
- [12] FreeRTOS, "FreeRTOS kernel documentation overview," accessed Jul. 15, 2026.
- [13] FreeRTOS, "FreeRTOS Kernel Documentation," accessed Jul. 15, 2026.
- [14] FreeRTOS, "Direct-to-task notifications," FreeRTOS Kernel Documentation, accessed Jul. 15, 2026.
- [15] R. Barry, *Mastering the FreeRTOS Real Time Kernel: A Hands-On Tutorial Guide*. Real Time Engineers Ltd.
- [16] J. Teich, "Hardware/software codesign: The past, the present, and predicting the future," *Proc. IEEE*, vol. 100, pp. 1411–1430, May 2012.
- [17] *IEEE Standard for VHDL Language Reference Manual*, IEEE Std 1076-2019, 2019.
- [18] AMD, "Vivado Design Suite User Guide: Logic Simulation (UG900)," 2026.
- [19] WHATWG, "Server-sent events," HTML Living Standard, sec. 9.2, accessed Jul. 15, 2026.

TABLE II
SELECTED CROSS-LAYER TRACEABILITY

Requirement	System model	UPPAAL	C/FreeRTOS	VHDL/Test
No entry without permission	Activity/sequence grant flow	Permission property	Task waits for scheduler grant	–
No conflicting concurrent movements	Conflict-check activity	Safety property	Core conflict check	ROM lookup
FIFS ordering	Activity and sequence models	Request ordering	Queue/core state	–
Decision within 50 ms	Requirement and server flow	Decision clock/property	Runtime timing check	Fast deterministic lookup
Backup before grant	Sequence ordering	Sync/ack behaviour	Sync and acknowledgement queues	–
Maximum channel occupancy	Scheduling model	Occupancy property	Configured channel count	–

TABLE III
MAIN PROJECT CONTRIBUTIONS AND PAPER-WRITING RESPONSIBILITIES

Team member	Main project contributions	Main paper sections drafted/reviewed	Target writing
Md Tawhidur Rahman Tuhin	Requirements analysis, UML/SysML-style system modelling, use-case and behavioural models, conflict-matrix discussions, testing, debugging, joint design decisions.	Introduction and objectives; technical background; requirements engineering; use-case model; activity model; sequence model.	3–4 pages
Moshiur Rahman Prince	UPPAAL timed-automata modelling, formal verification, timing and safety properties, queueing-model discussions, conflict-matrix validation, testing, debugging, joint design decisions.	UPPAAL formal model; formal verification; one-channel and two-channel queueing strategy; textual conflict matrix and its use across the project.	3–4 pages
Md Ratul Ahmed Alvi	Portable C scheduling core, FreeRTOS task architecture, live browser demonstration, HW/SW co-design, VHDL conflict ROM, ModelSim/QuestaSim verification, testing, debugging, joint design decisions.	C and FreeRTOS architecture; live demonstration; HW/SW co-design; VHDL and ModelSim verification; testing strategy.	3–4 pages

DECLARATION OF AUTHORSHIP

I, Moshiur Rahman Prince, herewith declare that I have composed the present paper and work by myself and without the use of any other than the cited sources and aids. Sentences or parts of sentences quoted literally are marked as such; other references with regard to the statement and scope are indicated by full details of the publications concerned. The paper and work in the same or similar form have not been submitted to any examination body and have not been published. This paper was not yet, even in part, used in another examination or as a course performance. I agree that my work may be checked by a plagiarism checker.

17.07.2026 Lippstadt - Moshiur Rahman Prince

I, Md Ratul Ahmed Alvi, herewith declare that I have composed the present paper and work by myself and without the use of any other than the cited sources and aids. Sentences or parts of sentences quoted literally are marked as such; other references with regard to the statement and scope are indicated by full details of the publications concerned. The paper and work in the same or similar form have not been submitted to any examination body and have not been published. This paper was not yet, even in part, used in another examination or as a course performance. I agree that my work may be checked by a plagiarism checker.

17.07.2026 Lippstadt - Md Ratul Ahmed Alvi

I, Md. Tawhidur Rahman Tuhin, herewith declare that I have composed the present paper and work by myself and without the use of any other than the cited sources and aids. Sentences or parts of sentences quoted literally are marked as such; other references with regard to the statement and scope are indicated

by full details of the publications concerned. The paper and work in the same or similar form have not been submitted to any examination body and have not been published. This paper was not yet, even in part, used in another examination or as a course performance. I agree that my work may be checked by a plagiarism checker.

17.07.2026 Lippstadt - Md. Tawhidur Rahman Tuhin

Note: It is important to note that all group members have worked on the different parts of this project. As a group, we always chose the best solution of the suggested ones by each member and collaborate to improve it.